

Technical Report: Insta360 Walk-to-Floorplan Pipeline

From Raw IMU/VIO to StellaVSLAM + SAM3 Roof Masking + Automated Alignment

Date: August 2026 **Project:** insta-360-imu

Table of Contents

1. [Executive Summary](#)
 2. [Ground Zero: Problem Statement & Input Data](#)
 3. [Phase 1: IMU-Only Pedestrian Dead Reckoning \(PDR\)](#)
 4. [Phase 2: Classical Visual-Inertial Odometry \(VIO\)](#)
 5. [Phase 3: MAST3R Dense Stereo Reconstruction](#)
 6. [Phase 4: StellaVSLAM Dense Point Cloud \(No Roof Mask\)](#)
 7. [Phase 5: SAM3 Roof Masking](#)
 8. [Phase 6: StellaVSLAM with Roof Mask + BEV Extraction](#)
 9. [Phase 7: Manual Landmark Alignment \(Click-Align\)](#)
 10. [Phase 8: Automated Hull-Based Alignment](#)
 11. [Phase 9: Start-Pinned Alignment](#)
 12. [Summary of All Approaches & Lessons Learned](#)
 13. [Appendix: Pipeline Architecture & Scripts](#)
-

1. Executive Summary

This report documents the full evolution of a pipeline that converts an Insta360 360-degree walkthrough video of a construction site into a 2D floorplan-aligned trajectory and wall map. The work progressed through **nine phases**, starting from the simplest possible approach (raw IMU dead reckoning) and systematically escalating complexity as each approach revealed its limitations:

1. **IMU-only PDR** — gyro heading integration + step detection (drift, no map)

2. **Classical VIO** — optical flow speed + IMU heading, no optimisation (noisy, scale-dependent)
 3. **MASt3R dense stereo** — global alignment from image pairs (metric scale, but sparse + slow)
 4. **StellaVSLAM** — feature-based SLAM with equirectangular input (dense map, but ceiling noise)
 5. **SAM3 roof masking** — text-prompted ceiling segmentation to clean the point cloud
 6. **StellaVSLAM + roof mask** — clean BEV wall map
 7. **Manual 3-click alignment** — Sim(2) transform via user-selected correspondences
 8. **Hull auto-alignment** — convex hull matching + distance transform scan matching (failed)
 9. **Start-pinned alignment** — 1 click + geometric optimisation for scale and rotation
-

2. Ground Zero: Problem Statement & Input Data

2.1 Goal

Given: - An **Insta360 X3** 360-degree video of a person walking through a construction site - The corresponding **IMU data** (accelerometer + gyroscope) embedded in the INSV container - A **2D architectural floorplan** (rasterised PNG) of the same building

Produce: - A **camera trajectory** overlaid on the floorplan, showing where the person walked - A **2D wall map** aligned to the floorplan, showing detected walls/obstacles

2.2 Input Data

- **Video:** Insta360 INSV container with dual-fisheye stitched to equirectangular (1920x960), ~2 minutes, ~30 fps
- **IMU:** 200 Hz 6-axis (3-axis accelerometer + 3-axis gyroscope), extracted from the INSV trailer binary format
- **Floorplan:** `floorplan-gf-maaksons.png` — rasterised architectural drawing of the ground floor
- **Environment:** Active construction site — missing outer walls, scaffolding, construction debris, partial interior walls

2.3 INSV Binary Parsing

Before any odometry could begin, the IMU data had to be extracted from the Insta360 proprietary INSV container format.

Script: `src/insv.py`

The INSV file stores metadata in a trailer at the end of the file. The parser: 1. Reads the trailer length from the last 4 bytes 2. Parses tagged records (type ID + length + payload) 3. Extracts IMU record type `0x03` / `0x300` — 20-byte packed samples containing timestamp, 3-axis accel, 3-axis gyro 4. Extracts exposure records (type `0x04` / `0x400`) for frame timestamp correlation

3. Phase 1: IMU-Only Pedestrian Dead Reckoning (PDR)

3.1 What

The very first approach: use only the Insta360's embedded IMU to estimate the walk trajectory. No video processing at all.

Script: `src/pdr.py`, `src/run_walk.py`

3.2 How

The PDR pipeline has two components:

Heading estimation (`heading_from_gyro`): 1. Low-pass filter the accelerometer to estimate the gravity vector direction 2. Subtract gyro bias (estimated from the lowest-variance window, assumed to be a standing-still period) 3. Project the gyroscope onto the gravity axis to extract yaw rate 4. Integrate yaw rate over time to get heading: $\text{yaw}(t) = \int \text{gyro_gravity}(t) dt$

Step detection and dead reckoning (`detect_steps`, `dead_reckon`): 1. Compute accelerometer magnitude, bandpass filter (0–3 Hz) 2. Detect peaks in the filtered signal (each peak = one walking step) 3. Assign a fixed step length (default 0.70m) to each detected step 4. Between steps, position is propagated using: $x += \text{step_length} * \cos(\text{yaw})$, $y += \text{step_length} * \sin(\text{yaw})$

Floorplan overlay: The PDR trajectory was scaled to fit the floorplan image dimensions (stretch-to-fit, no alignment).

3.3 Why This Approach First

- **Simplest possible baseline** — requires no video processing, no feature extraction, no optimisation
- Tests whether the IMU data quality is sufficient for heading estimation
- Validates the INSV binary parser and IMU extraction

3.4 Theoretical Basis

Pedestrian dead reckoning using inertial sensors is well-established (Foxlin 2005, Harle 2013). The key insight: for pedestrians, step detection from accelerometer peaks is more reliable than double-

integrating acceleration (which drifts quadratically). Heading from gyro integration drifts linearly, but is acceptable for short walks (~2 minutes).

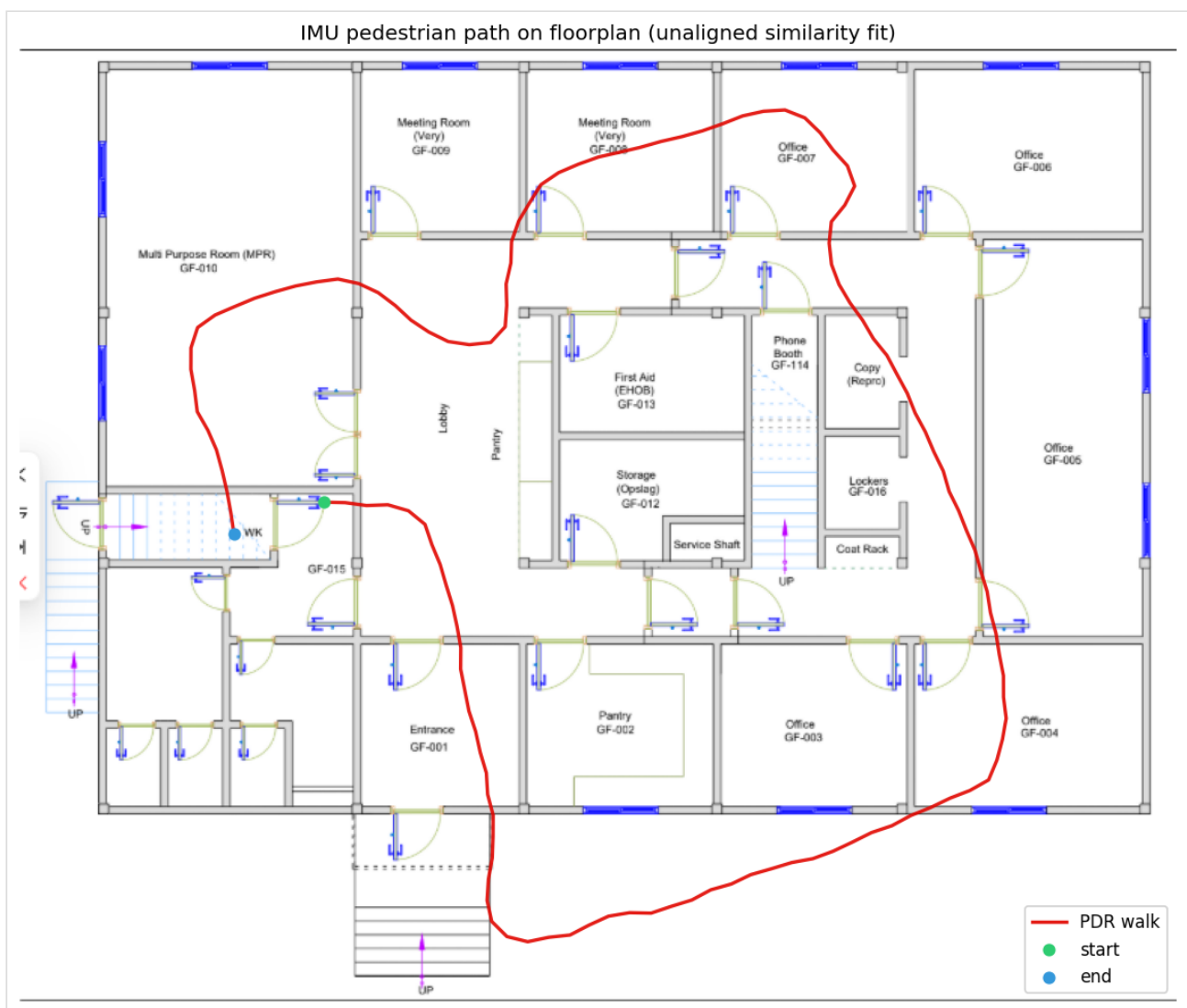
3.5 Results

Output: `outputs/VID_20260810_124225_00_064/`

- `imu_pdr.png` — IMU signals and PDR trajectory plot
- `trajectory_on_floorplan.png` — PDR path overlaid on floorplan (unaligned stretch-to-fit)

Key metrics (from `summary.json`): - Camera: Insta360 X4 Air, firmware v1.2.7 - IMU: 80,192 samples over 80s @ 1001 Hz - PDR: **129 steps** detected, step length 0.70m, total path **90.3 m** - 40 keyframes extracted at 2s intervals

The PDR trajectory showed the general shape of the walk (corridor traversals, turns), but with significant heading drift and no metric-to-pixel alignment.



3.6 Problems

1. **Heading drift:** Gyro integration accumulates error. Over 2 minutes, heading error grows to several degrees, causing the trajectory to diverge from the true path.
 2. **Fixed step length:** Not all steps are the same length. Turns, slow-downs, and variable gait introduce position error.
 3. **No map/occupancy:** PDR produces only a trajectory, no wall or obstacle information.
 4. **No alignment:** The trajectory is in an arbitrary IMU coordinate frame with no scale, rotation, or translation relationship to the floorplan.
 5. **No loop closure:** If the walk revisits an area, PDR has no way to correct accumulated drift.
-

4. Phase 2: Classical Visual-Inertial Odometry (VIO)

4.1 What

Combine IMU heading with visual motion estimation from the 360-degree video. This is still a simple, filter-less approach — no Kalman filter, no bundle adjustment, no graph optimisation.

Scripts: `src/vio.py`, `src/backend.py`, `src/run_align.py`

4.2 How

The `ClassicalVIOBackend` processes the video at ~8 fps (every 4th frame):

Visual speed estimation: 1. Detect Shi-Tomasi features (`cv2.goodFeaturesToTrack`) in each grayscale frame (max 350 corners) 2. Track features to the next frame using Lucas-Kanade optical flow (`cv2.calcOpticalFlowPyrLK`) 3. Compute the median optical flow magnitude as raw visual speed in pixels 4. Subtract estimated rotation-induced flow: `trans_px = vis_px - 0.6 * rot_px` (where `rot_px = |Δyaw| * fx`) 5. Convert pixel speed to metric speed: `v_vis = clip((trans_px / dt) * 0.045, 0, 2.4)`

Speed fusion with IMU steps: - If steps are detected in the current interval: `speed = 0.35 * v_vis + 0.65 * v_step` - If no steps but visual speed is significant (> 0.28 m/s): `speed = v_vis` - Otherwise: `speed = 0` (standing still)

Position integration: - `x += speed * dt * cos(yaw_imu)` - `y += speed * dt * sin(yaw_imu)`

Ground-plane point projection: - Tracked features in the lower half of the image are back-projected onto a ground plane at $z=0$ (assuming camera height = 1.70m) - These 2D "ground points" form a sparse occupancy map of obstacles visible at floor level

Occupancy grid: - Ground points are binned into a 0.25m grid - Morphologically dilated for continuity - This is the first attempt at a "wall map"

4.3 Floorplan Alignment

Scripts: `src/align.py`, `src/click_align.py`, `src/run_align.py`

The alignment used **timestamp-based landmarks**: the user identifies points on the floorplan and specifies the *time* in the video when the walker was at that point.

- Each landmark = `(t_s, px, py)` : "at time `t_s` seconds, the camera was at pixel `(px, py)` on the floorplan"
- From `t_s`, the world-frame position is interpolated from the VIO trajectory
- With 2+ correspondences, a Sim(2) (Umeyama) transform is computed mapping VIO coordinates to floorplan pixels
- For 2 points exactly, a direct two-point similarity is used; for 3+, least-squares Umeyama

The alignment also evaluated both reflection modes (mirrored vs normal) and optionally scored using free-space (trajectory should not cross walls).

4.4 Why This Approach

- **Adds visual information** to pure IMU: optical flow gives translational speed independent of step detection
- **Still simple**: No SLAM infrastructure, no optimisation, no loop closure — just forward integration
- **Ground-point projection** gives a first rough occupancy map, even if crude
- Tests whether visual features from equirectangular images are trackable

4.5 Theoretical Basis

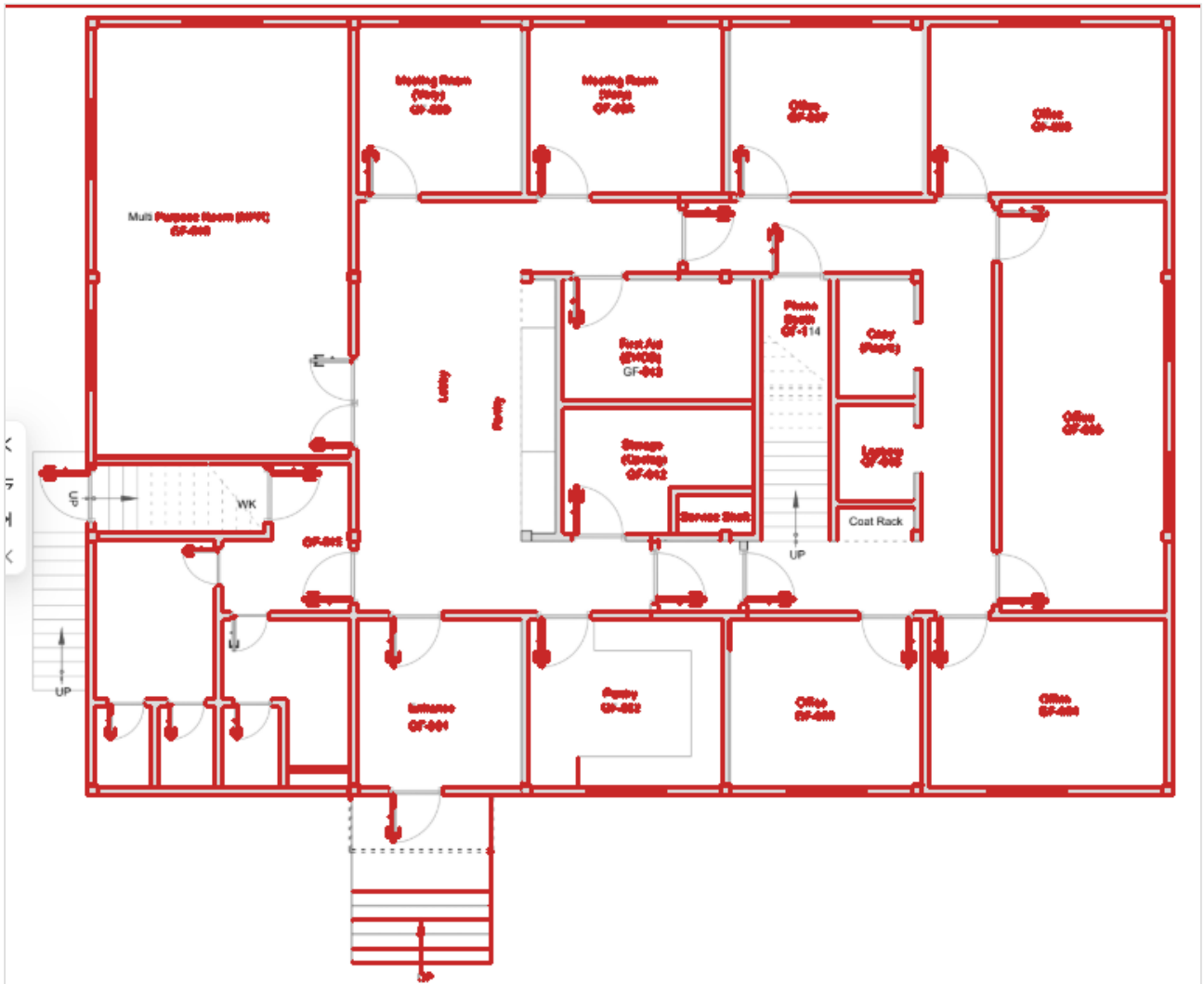
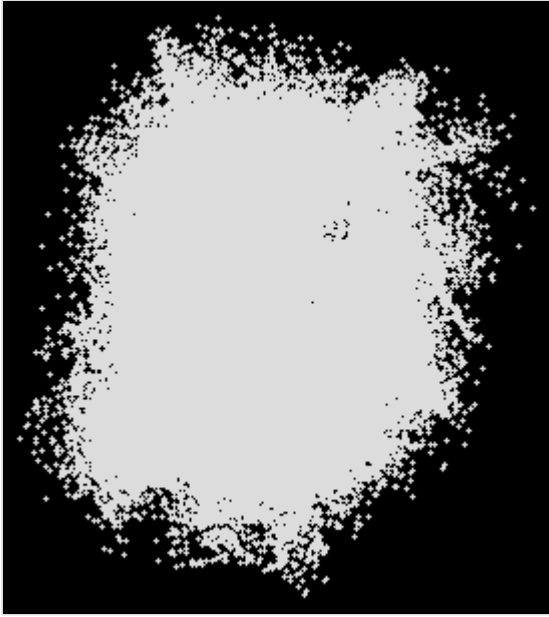
This is a degenerate VIO: it uses the gyro for heading and optical flow for speed, but never fuses them in a probabilistic filter. The heading is purely IMU-derived (no visual rotation estimation). The speed is a weighted blend of visual and step-based estimates, with hand-tuned weights.

The 0.045 px-to-m conversion factor (`v_vis = trans_px/dt * 0.045`) is an empirical calibration that assumes a fixed relationship between optical flow magnitude and real-world speed for a forward-facing equirectangular camera at ~1.7m height. This is inherently approximate.

4.6 Results

Output: `outputs/run2-capture/`

Key metrics (from `align_summary.json`): - Backend: `classical_vio` - 899 frames processed (stride 4 from 29.97 fps video) - Path length: **128.65 m** - 165,825 ground points projected -



4.7 Problems

1. **Negative free-space score:** The aligned trajectory passes through walls frequently, indicating poor alignment accuracy. A score of -36.4 means the trajectory is mostly off-floorplan or on wall pixels.
2. **Scale is reflected:** The best alignment used `reflected=true`, suggesting the VIO coordinate frame is mirrored relative to the floorplan — a sign of ambiguity in the visual speed estimation.
3. **Noisy occupancy:** The 165k ground points produce a very noisy occupancy grid. The crude ground-plane assumption (fixed camera height, pinhole approximation on equirectangular images) causes mis-projections.
4. **No loop closure:** Over 128m of walking, heading drift accumulates significantly. The trajectory shape deviates from the true path.
5. **Hand-tuned parameters:** The 0.045 px-to-m factor, 0.35/0.65 visual/step weights, and 0.6 rotation compensation are all hand-tuned and fragile.
6. **Scale ambiguity:** Visual optical flow cannot distinguish a 1m displacement at 2m distance from a 2m displacement at 4m distance. The step length provides some metric grounding, but is itself approximate.

4.8 Why It Was Abandoned

The classical VIO produced a trajectory that was recognisably a walk through the building, but the alignment quality was poor (negative free-space, reflected, noisy walls). The fundamental problem: without optimisation (bundle adjustment, loop closure), errors accumulate over the ~2-minute walk and the trajectory drifts too far for accurate floorplan overlay.

5. Phase 3: MAST3R Dense Stereo Reconstruction

5.1 What

Replace the hand-crafted VIO with **MASt3R** (Matching And Stereo TRansformer) — a deep learning model that performs dense stereo matching and jointly estimates camera poses via sparse global alignment.

Scripts: `src/mast3r_odom.py`, `src/mast3r_backend.py`

5.2 How

1. **Frame extraction** (`extract_frames`): Sample ~50 frames uniformly across the walk at ~2 fps. Resize to max 960px. Save as JPEGs.

2. **Pair generation:** Create image pairs using a sliding window (`swin-5-noncyclic`): each frame is matched with the next 5 frames. Pairs are symmetrised ($A \rightarrow B$ and $B \rightarrow A$).
3. **MASt3R inference** (`run_mast3r_sga`):
4. Load `n timer/MASt3R_ViT Large_BaseDecoder_512_catmlpdpt_metric` pretrained model
5. Run on all pairs to compute dense correspondences and relative poses
6. Perform **sparse global alignment** (SGA): jointly optimise all camera poses to be globally consistent
7. SGA parameters: `lr1=0.07, niter1=200, lr2=0.01, niter2=100`
8. **Pose extraction** (`_poses_to_xy_yaw`):
9. Extract camera-to-world transforms (4x4 matrices)
10. Project camera centres to XZ ground plane (Y-up convention)
11. Compute yaw from the camera forward vector projected to ground
12. **Alignment:** Same Sim(2) landmark-based alignment as the VIO approach.

5.3 Why MASt3R

- **Metric scale:** MASt3R is trained on metric datasets and outputs metric camera poses, eliminating the scale ambiguity that plagued classical VIO
- **Global optimisation:** SGA jointly optimises all camera poses, providing implicit loop closure
- **No hand-tuned parameters:** The deep model handles feature matching, outlier rejection, and pose estimation end-to-end
- **Dense correspondences:** Unlike sparse ORB/SIFT matching, MASt3R produces dense pixel-wise correspondences

5.4 Theoretical Basis

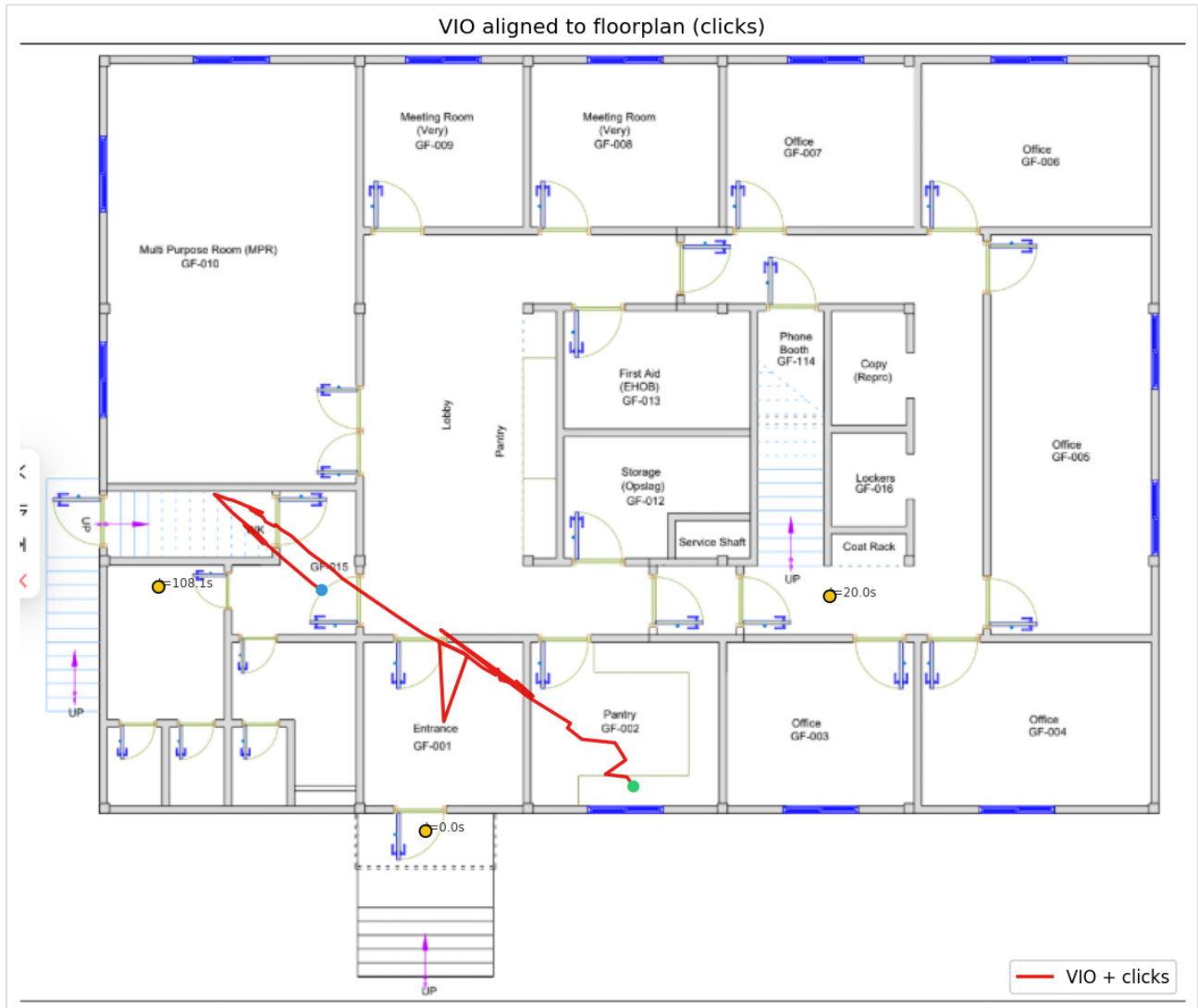
MASt3R (Leroy et al., 2024) extends DUS t3R by adding metric scale supervision. The sparse global alignment formulation is equivalent to a pose-graph optimisation where each pair constraint is a relative pose estimated by the transformer network. The two-stage optimisation (coarse lr_1 then fine lr_2) helps escape local minima while converging to a consistent global solution.

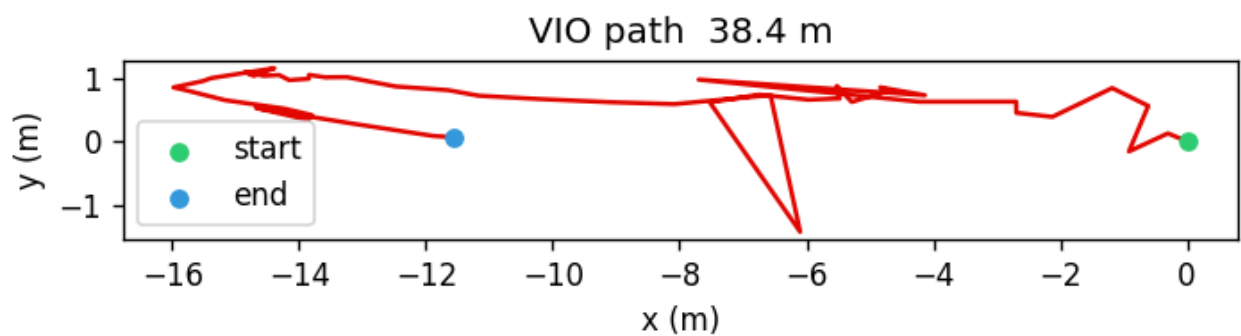
5.5 Results

Output: `outputs/run2-capture-mast3r/`

Key files: - `trajectory_vio.png` — Raw MASt3R trajectory (world frame) - `trajectory_aligned.png` — Aligned to floorplan - `occupancy.png` — Sparse occupancy from camera positions - `mast3r_cams2world.npy` — 50 camera-to-world 4x4 matrices - `vio_meta.json` — Backend metadata

Key metrics (from `vio_meta.json` and `align_summary.json`): - 50 frames at 2 fps, CUDA device - Path length: **38.4 m** (vs 128.6m for classical VIO — likely more accurate, VIO over-estimated) - Net displacement: **11.6 m** - Alignment: 3 landmarks, scale 21.03 px/m, rotation 31.77°, reflected=true - Free-space score: **-155.67** (worse than classical VIO — trajectory heavily off-floorplan) - Occupancy grid: 28×81 cells at 0.25m (very sparse — only camera positions, not dense points)





5.6 Problems

1. **Sparse trajectory:** Only ~50 frames means ~50 camera positions. The trajectory is interpolated between them but lacks the density of per-frame VIO.
2. **Slow:** MAST3R + SGA on 50 frames takes significant GPU time (minutes on a single GPU). Not suitable for real-time or iterative development.
3. **No dense occupancy:** MAST3R gives camera poses but not a dense 3D point cloud (unlike SLAM). The "occupancy" is just camera positions — no wall map.
4. **Equirectangular issues:** MAST3R was trained on perspective images. Applying it to equirectangular frames (or naive crops) may not leverage its full accuracy.
5. **Memory intensive:** The ViT-Large model + dense pair matching requires significant GPU memory, limiting max_frames.

5.7 Why It Was Moved On From

MASt3R provided better metric scale than classical VIO, but the sparse trajectory (50 points vs 900) and lack of a dense map made it insufficient for wall detection and BEV generation. The pipeline needed a system that produces both a dense trajectory AND a dense 3D point cloud. This led to adopting StellaVSLAM.

6. Phase 4: StellaVSLAM Dense Point Cloud (No Roof Mask)

6.1 What

Use **StellaVSLAM** — a feature-based visual SLAM system that natively supports equirectangular input — to produce a dense 3D point cloud and camera trajectory.

Scripts: `pipeline/src/01_cubemap_crop.py`, `pipeline/src/02_stella_slam.py`

6.2 How

StellaVSLAM (formerly OpenVSLAM) runs in a Docker container:

1. Convert the 360-degree video to 1920x960 equirectangular MP4
2. Run StellaVSLAM in dense mode with ORB features
3. StellaVSLAM performs:
 4. ORB feature detection and matching across frames
 5. Local and global bundle adjustment
 6. Keyframe selection and map point triangulation
 7. Loop closure detection via bag-of-words (ORB vocabulary)
8. Outputs: `out.ply` (3D point cloud), `out.db` (keyframe database), frame trajectory

6.3 BEV Extraction

The 3D point cloud is projected into a 2D Bird's Eye View:

1. Filter points by height: keep `z ∈ [0.3m, 2.5m]` (above floor, below ceiling)
2. Project onto XY plane
3. Bin into 0.05m grid
4. Apply occupancy threshold
5. Detect wall lines via Hough transform

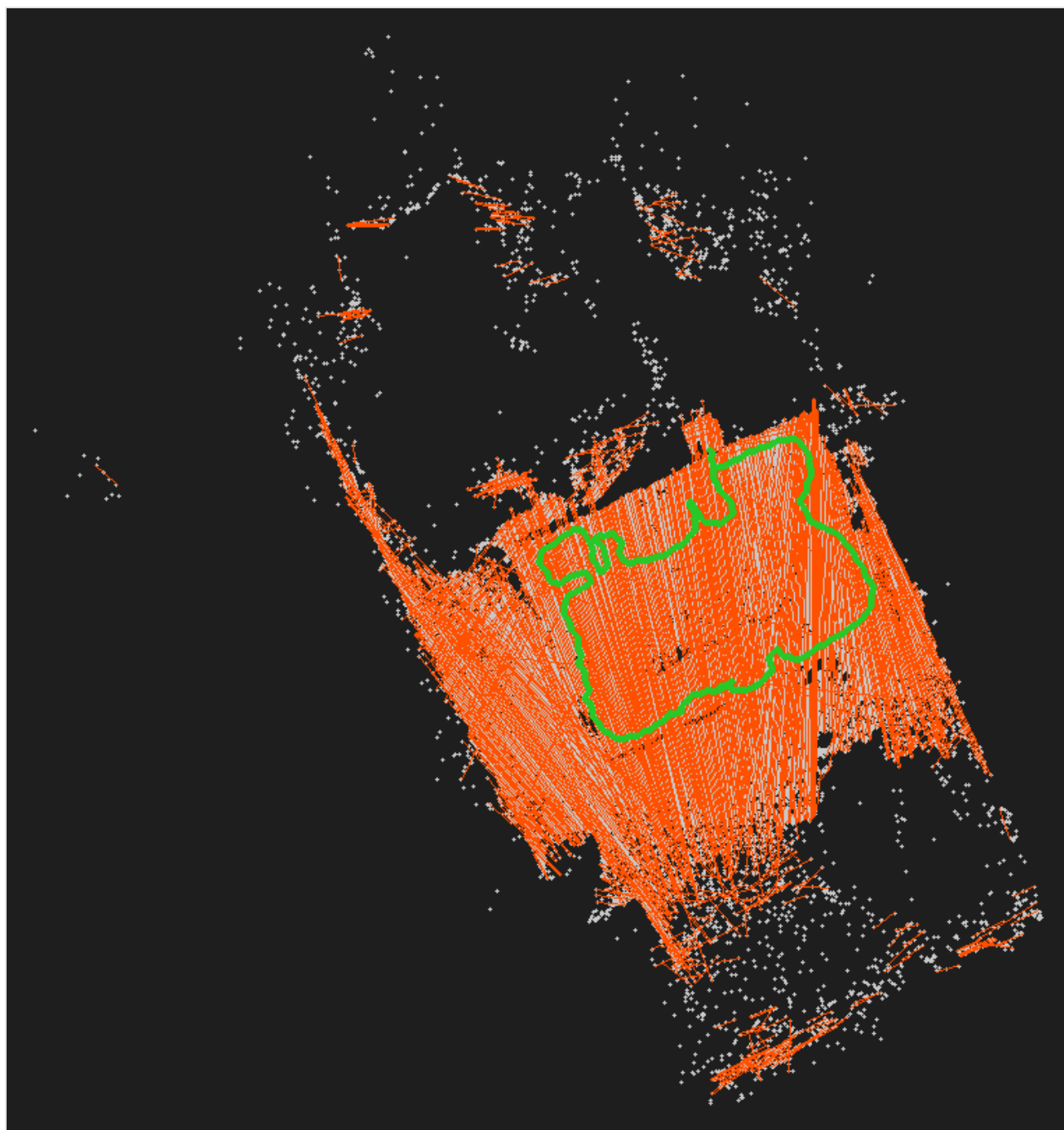
6.4 Why StellaVSLAM

- **Equirectangular native:** Unlike MAST3R (perspective-trained), StellaVSLAM has a dedicated equirectangular camera model — no cubemap decomposition needed for SLAM itself
- **Dense map:** Produces thousands of 3D map points, giving rich BEV occupancy
- **Real-time capable:** Designed for real-time operation; processes a 2-minute video in reasonable time
- **Loop closure:** Built-in loop closure via BoW reduces trajectory drift
- **Mature system:** Well-tested SLAM framework with known behaviour

6.5 Results (Without Roof Mask)

Output: `pipeline/outputs/run2-stella/`

Key metrics (from `bev_summary.json`): - 500,000 points sampled - 294,334 in the wall-height band - **830 Hough lines** detected - 1,788 trajectory samples - Roof mask fraction: 26.9% (old geometric-only approach)



7. Phase 5: SAM3 Roof Masking

7.1 What

Use Meta's SAM3 (Segment Anything Model 3) to identify and mask out roof/ceiling regions in each 360-degree frame *before* SLAM processing. By telling StellaVSLAM to ignore masked pixels, the feature detector will not extract ceiling features, and the resulting point cloud should contain only wall-level geometry.

Script: `pipeline/src/00_sam3_roof_mask.py`

7.2 Initial Implementation

1. Convert each equirectangular frame into a **cubemap** (6 pinhole camera faces: front, right, back, left, up, down)
2. Run SAM3 on the **"up" face only** (pointing at ceiling)
3. Apply a **geometric clip** (`horizon_top_frac = 0.62`) to the four horizon faces — painting the top 62% of each side face as "roof" without running SAM3 on them
4. Remap all face masks back to equirectangular coordinates to produce a unified Stella mask

Why cubemap decomposition for SAM3? SAM3 expects perspective images. Equirectangular images have extreme distortion at the poles, making segmentation unreliable. The cubemap faces are perspective-correct, giving SAM3 clean input.

7.3 Problem: Over-Masking

User observation: *"The images are just reddening out more than half of the image, even including areas that aren't roofs or ceilings."*

Debug overlays showed vast portions of horizon faces painted red, including walls, doorways, and vertical structure.

7.4 Root Cause Analysis

Two distinct issues:

1. **Geometric over-painting:** `horizon_top_frac = 0.62` painted the top 62% of each side face as roof regardless of content. This aggressively masked walls, scaffolding, and door frames.
2. **Misleading debug overlays:** The debug visualisation remapped the union mask back onto cubemap faces, making the SAM3-only up-face mask appear to bleed into side faces. The overlays didn't show actual per-face segmentation.

7.5 Fix: Per-Face SAM3 Prompting

Parameter	Before	After	Rationale
<code>horizon_top_frac</code>	0.62	0.0	Disable geometric painting entirely
SAM3 faces	Up only	Front, Right, Back, Left, Up	Run SAM3 on all 5 relevant faces
Horizon prompts	None	"corrugated roof", "metal roof", "roof"	Targeted ceiling terms for side views
Up prompts	"roof", "ceiling"	"ceiling", "roof"	More specific
Negative prompts	None	"sky", "tree" (horizon only)	Prevent masking exterior through missing walls
Confidence threshold	0.35	0.40	Reduce false positives
Bloat filter	None	<code>MAX_HORIZON_INSTANCE_FRAC</code>	Reject masks covering too much of a face
Y-centroid check	None	Mean Y < face_height/2	Only keep masks in upper half of horizon faces
Debug overlays	Remapped union	Per-face actual SAM3 mask	Show what SAM3 actually segmented

Key design decisions:

- **Negative prompts** ("sky" , "tree") were critical for the construction site: missing outer walls expose sky and trees, which SAM3 could confuse with overhead ceiling.
- **Bloated mask rejection** prevents SAM3 from returning a mask covering the entire face (happens with ambiguous prompts).
- **Y-centroid filtering** ensures only masks in the upper portion of horizon faces are kept — ceiling structure appears in the upper half of a forward-looking camera.

7.6 Results

Output: `pipeline/outputs/run2-stella-roofmask/roof_mask.png` ,
`roof_mask_overlay.jpg`

From `roof_mask_meta.json` : - 20 frames sampled, SAM3 hit all 20 - Per-face hits: front 18, right 18, back 17, left 19, up 14 - Roof fraction: **48.2%** (correct: more ceiling removed by proper per-face segmentation) - Mask source: `sam3_per_face`



8. Phase 6: StellaVSLAM with Roof Mask + BEV Extraction

8.1 What

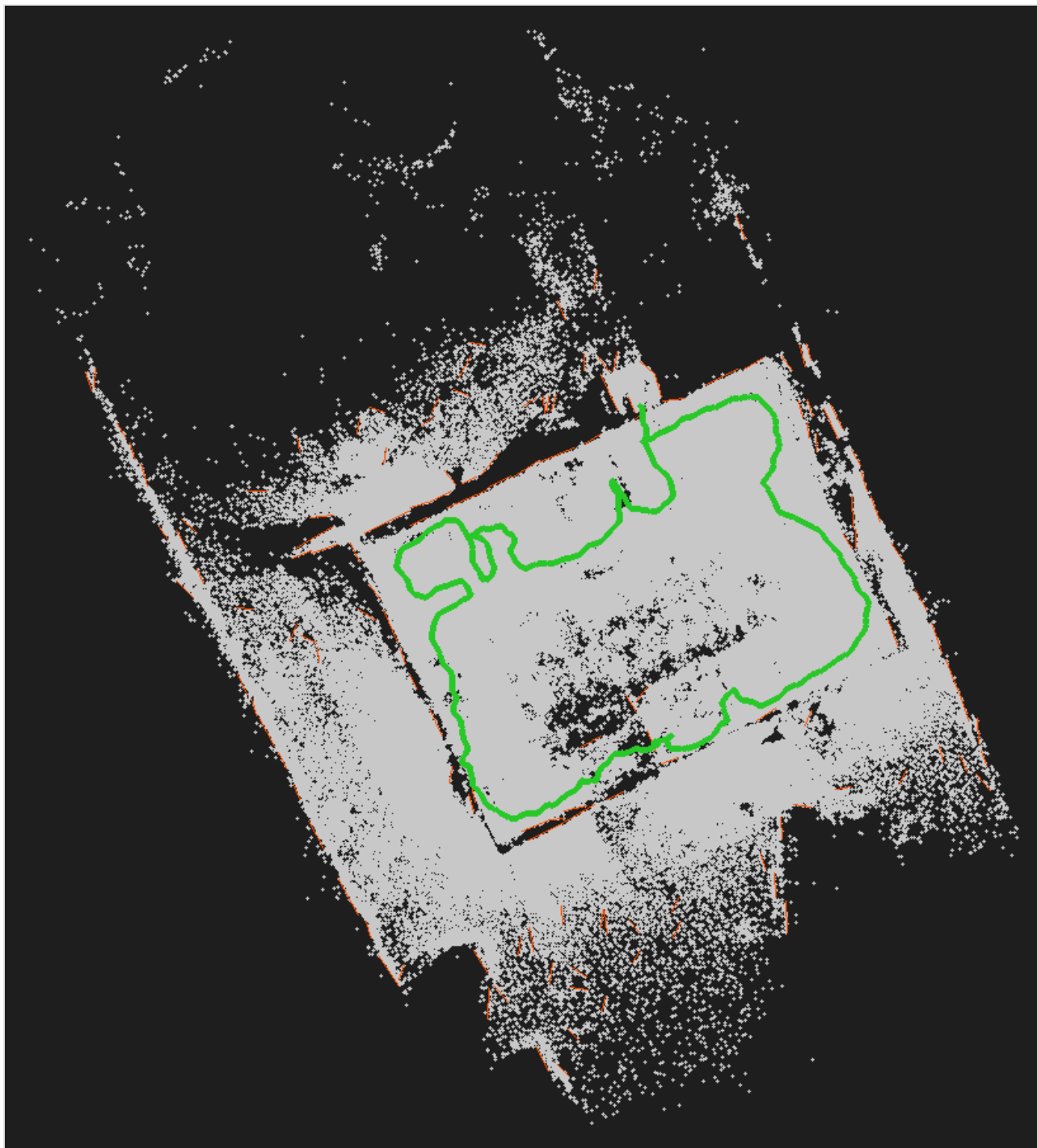
Re-run the full StellaVSLAM pipeline with the SAM3 roof mask applied. Stella ignores features in masked (white) regions.

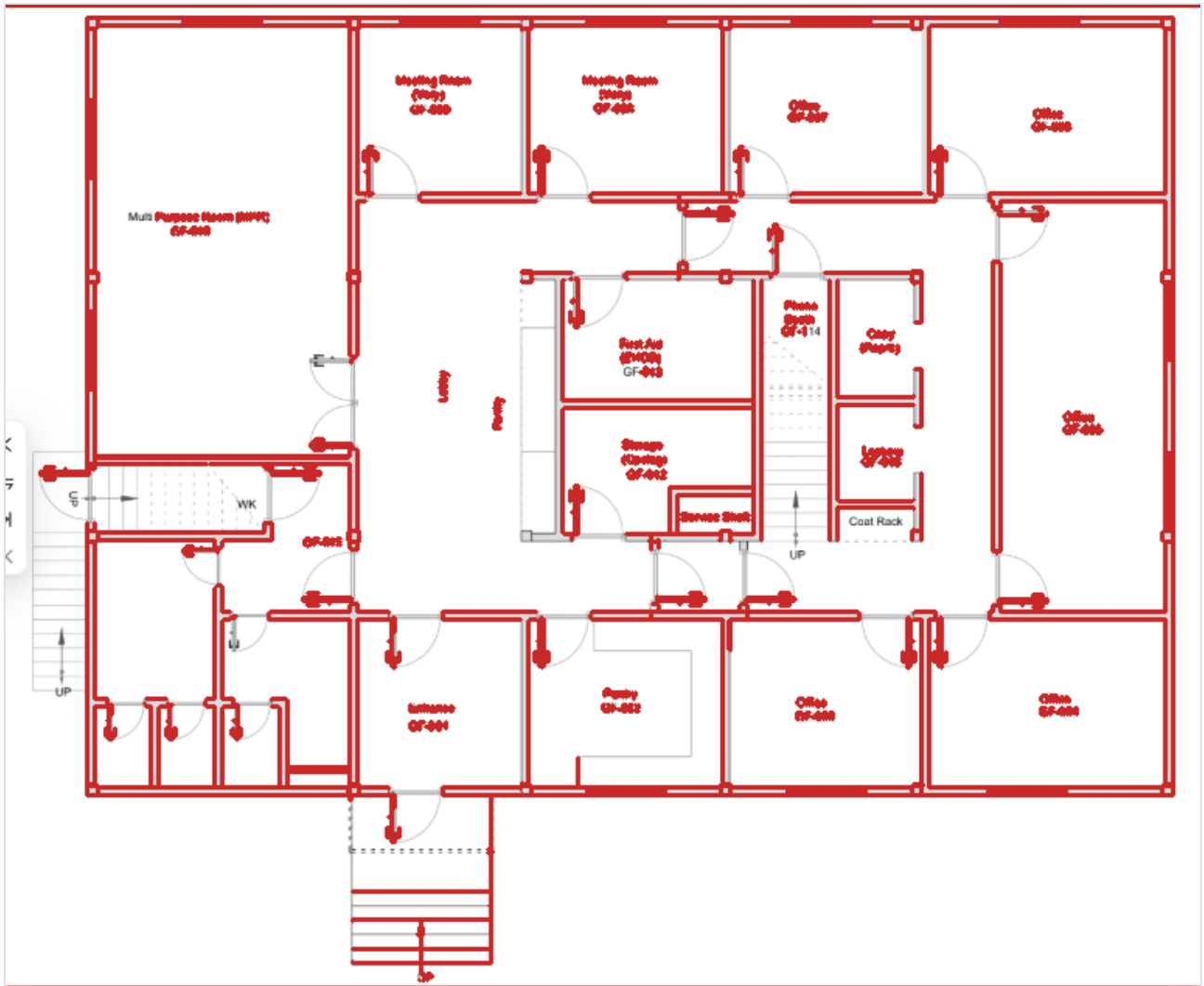
8.2 Results: Before vs After

Metric	Without Mask	With Mask
Points sampled	500,000	500,000
Wall-band points	294,334	477,616
Hough lines	830 (noisy)	143 (clean)
Roof fraction	26.9%	48.2%

Key observations: - Wall-band points increased from 294k to 478k — more actual wall features being detected - Hough lines dropped from 830 to 143 — far fewer spurious ceiling-projected lines - In CloudCompare, the point cloud showed the first floor / roof layer completely removed - BEV

showed clear wall outlines and corridor boundaries - Interior spaces appeared as open (free) with walls clearly delineated





9. Phase 7: Manual Landmark Alignment (Click-Align)

9.1 What

Align the clean BEV and trajectory to the floorplan using **3 user-selected point correspondences**.

Script: `pipeline/src/03_auto_align.py` (click UI), reusing concepts from `src/align.py`

9.2 How

The user clicks 3 points on the BEV image and the corresponding 3 points on the floorplan. Each pair maps a BEV pixel to a floorplan pixel. Unlike the earlier VIO alignment (which used timestamps), this maps BEV-space points directly to floorplan-space points.

From 3 correspondences, a **Sim(2) transform** is computed:

$$p_{\text{plan}} = s * R(\theta) * p_{\text{bev}} + t$$

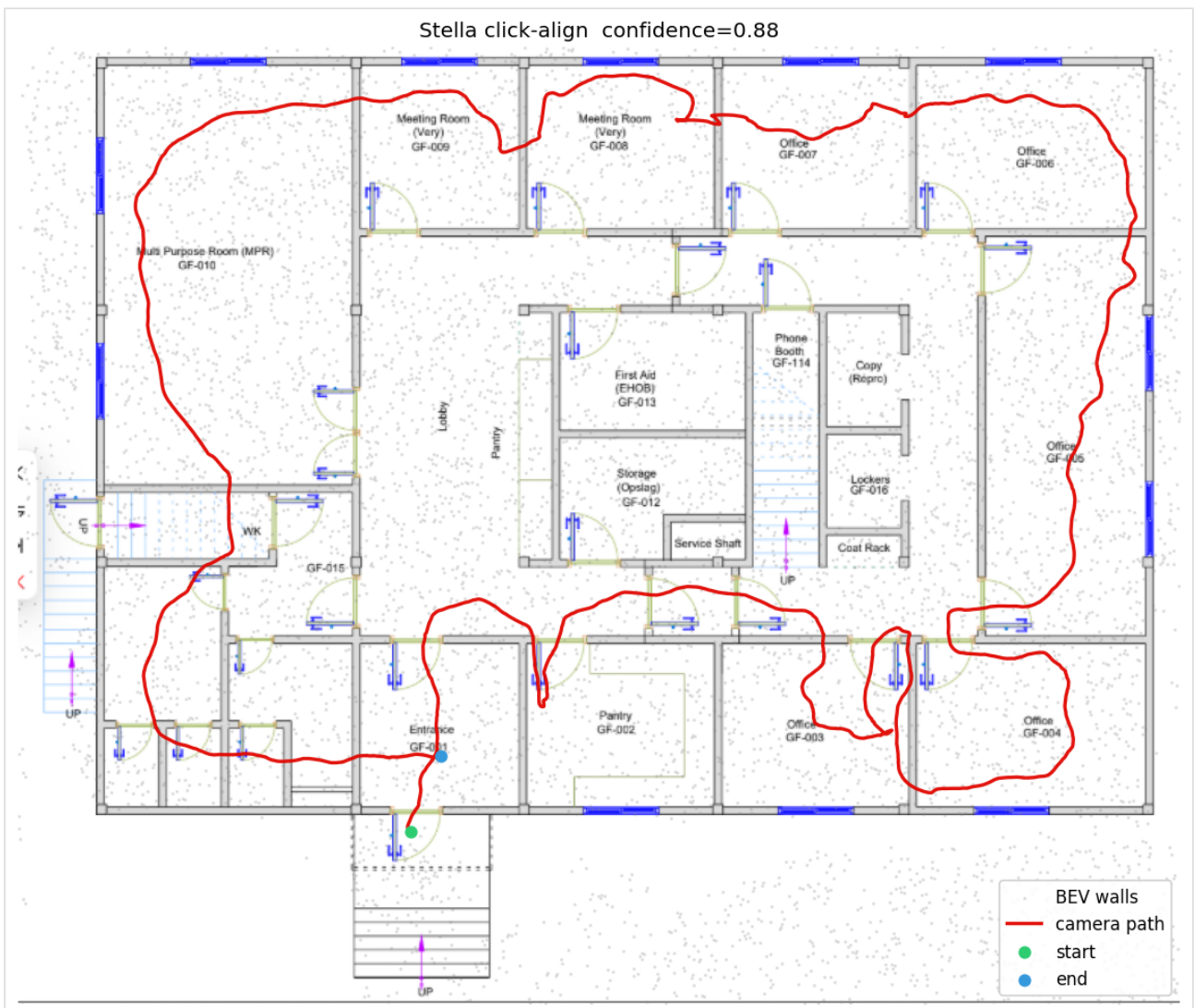
Where s = scale, $R(\theta)$ = rotation, t = translation. With 3 pairs (6 equations, 4 unknowns), the system is over-determined and solved via least-squares (Umeyama method).

9.3 Results

Output: pipeline/outputs/run2-stella-roofmask/

From `sim2_transform.json` and `align_summary.json` : - **Source:** click (3 correspondences) - **Scale:** 38.53 px/m - **Rotation:** -154.09° - **Translation:** (259.9, 519.1) px - **Reflected:** false - **Landmark RMSE:** 9.32 px - **Confidence:** 0.883 - **Free-space score:** 0.932 (93.2% of trajectory points on walkable area)

The 3 correspondences used (from `correspondences.json`) : - Pair 1: BEV (501.4, 316.5) → Plan (265.4, 525.0) - Pair 2: BEV (524.0, 391.5) → Plan (276.6, 357.8) - Pair 3: BEV (400.2, 640.9) → Plan (707.3, 43.2)



9.4 Limitations

1. **Manual effort:** Requires identifying 3 unambiguous corresponding points
 2. **Not scalable:** Every new capture needs new clicks
 3. **Sensitivity:** 9.32 px RMSE at click points; errors propagate away from landmarks
 4. **Requires recognisable features:** On a construction site BEV, finding clear correspondences is non-trivial
-

10. Phase 8: Automated Hull-Based Alignment

10.1 Motivation

User request: *"I need to figure out a way to remove the manual point selection thing somehow."*

Goal: fully automate the Sim(2) alignment with zero user clicks.

10.2 Method

Script: `pipeline/src/06_hull_scan_align.py`

Stage 1 — Hull Extraction: - Floorplan: binarise wall ink, strip page borders, compute convex hull
- BEV: identify main building blob (connected component containing most trajectory), compute convex hull

Stage 2 — Coarse Alignment: - Compute minimum-area bounding rectangles of both hulls -
Generate 8 Sim(2) hypotheses by aligning rectangles at 4 rotation offsets × 2 reflection modes

Stage 3 — Fine Alignment (Distance Transform Scan Matching): - Compute floorplan distance transform (every pixel → distance to nearest wall) - Score each hypothesis: chamfer (mean distance of BEV points to walls) + free-space + hull IoU - Local grid search refinement

10.3 Theoretical Justification

- **Convex hulls** approximate building footprints; matching them gives coarse scale/rotation/translation
- **Distance transform scan matching** (cf. Olson 2009) provides smooth cost surface for alignment refinement
- **Free-space scoring** regularises against scale collapse

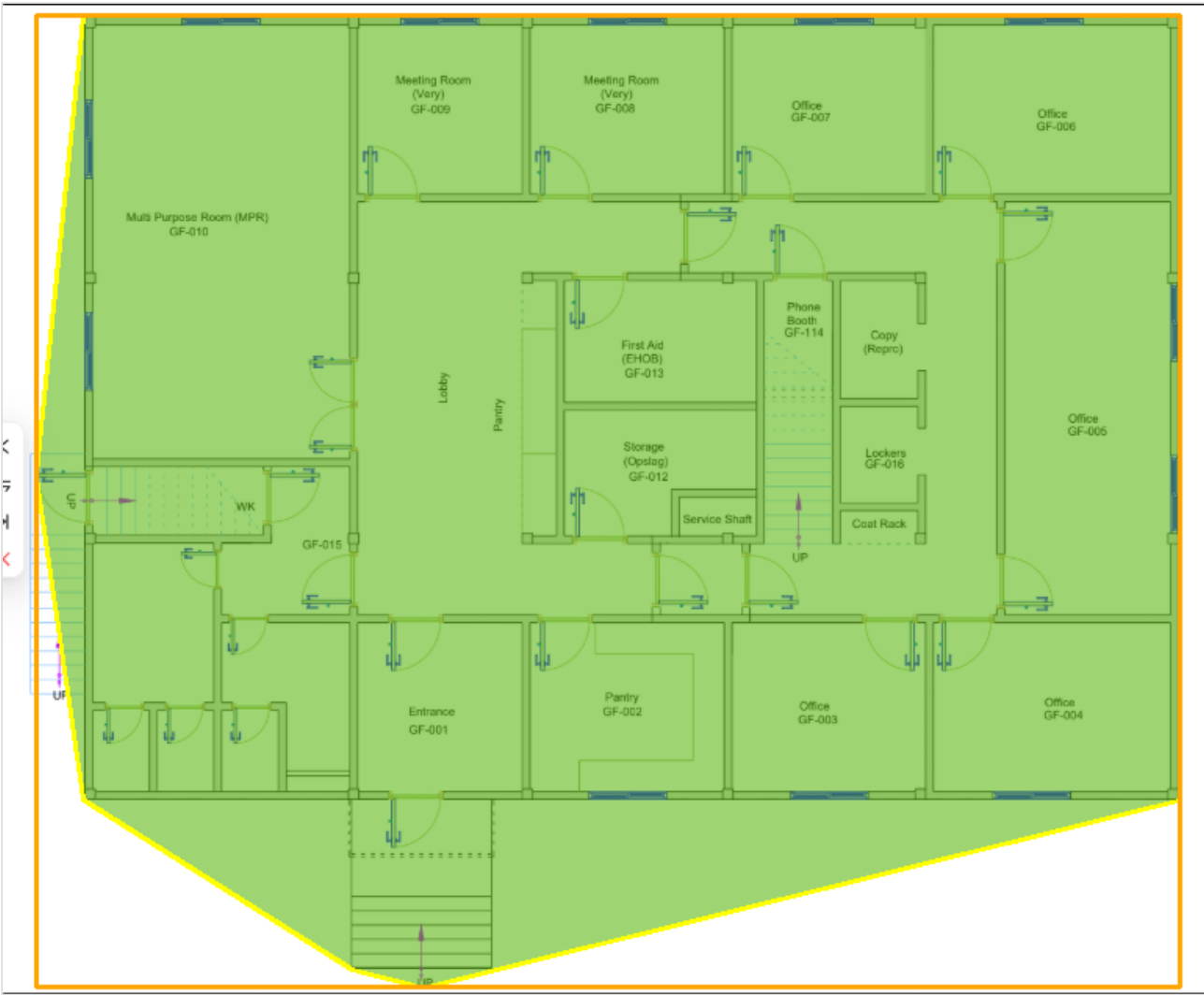
10.4 Results

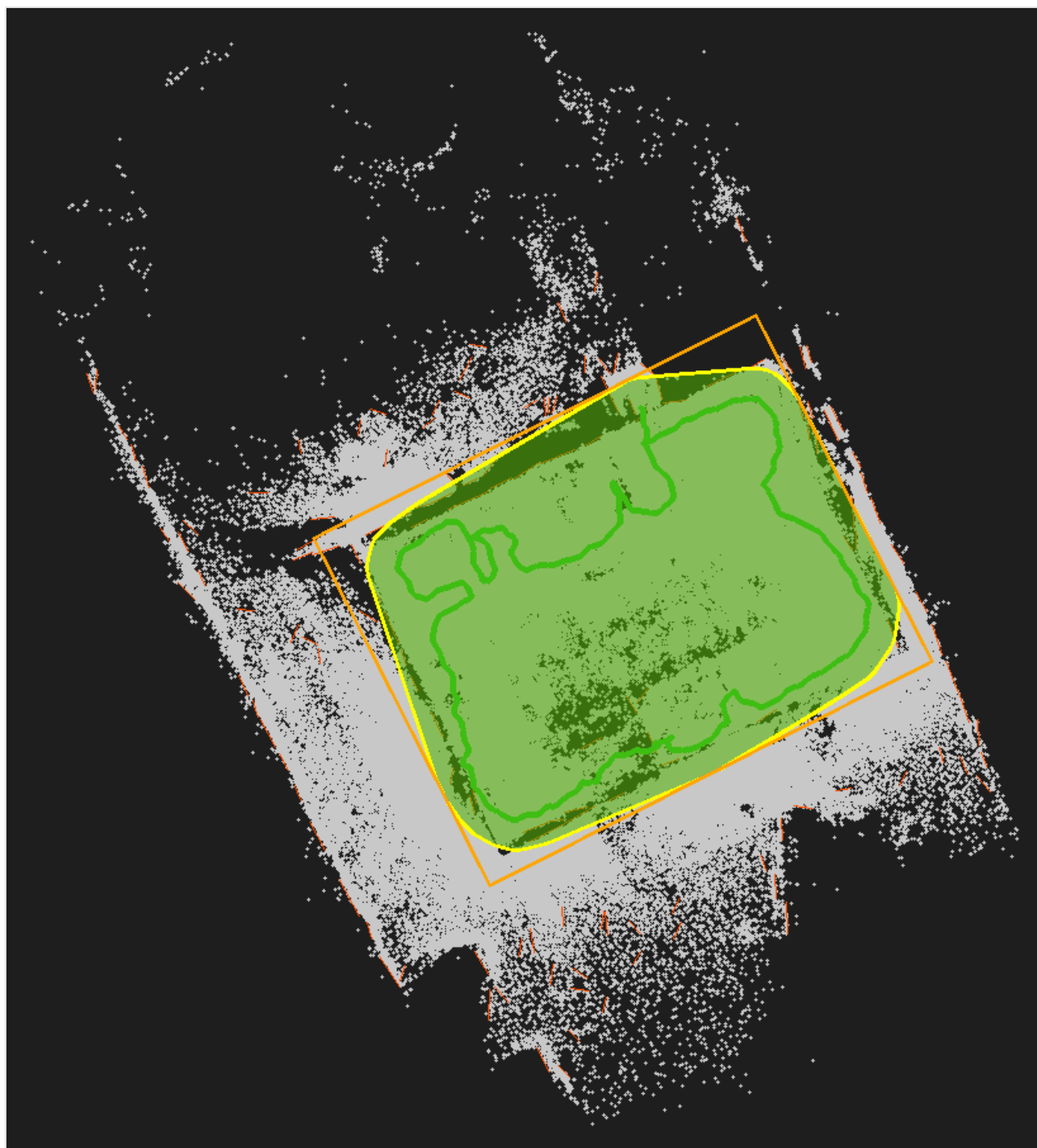
Output: `pipeline/outputs/run2-stella-roofmask/auto-align/`

From `summary.json` , compared to 3-click reference:

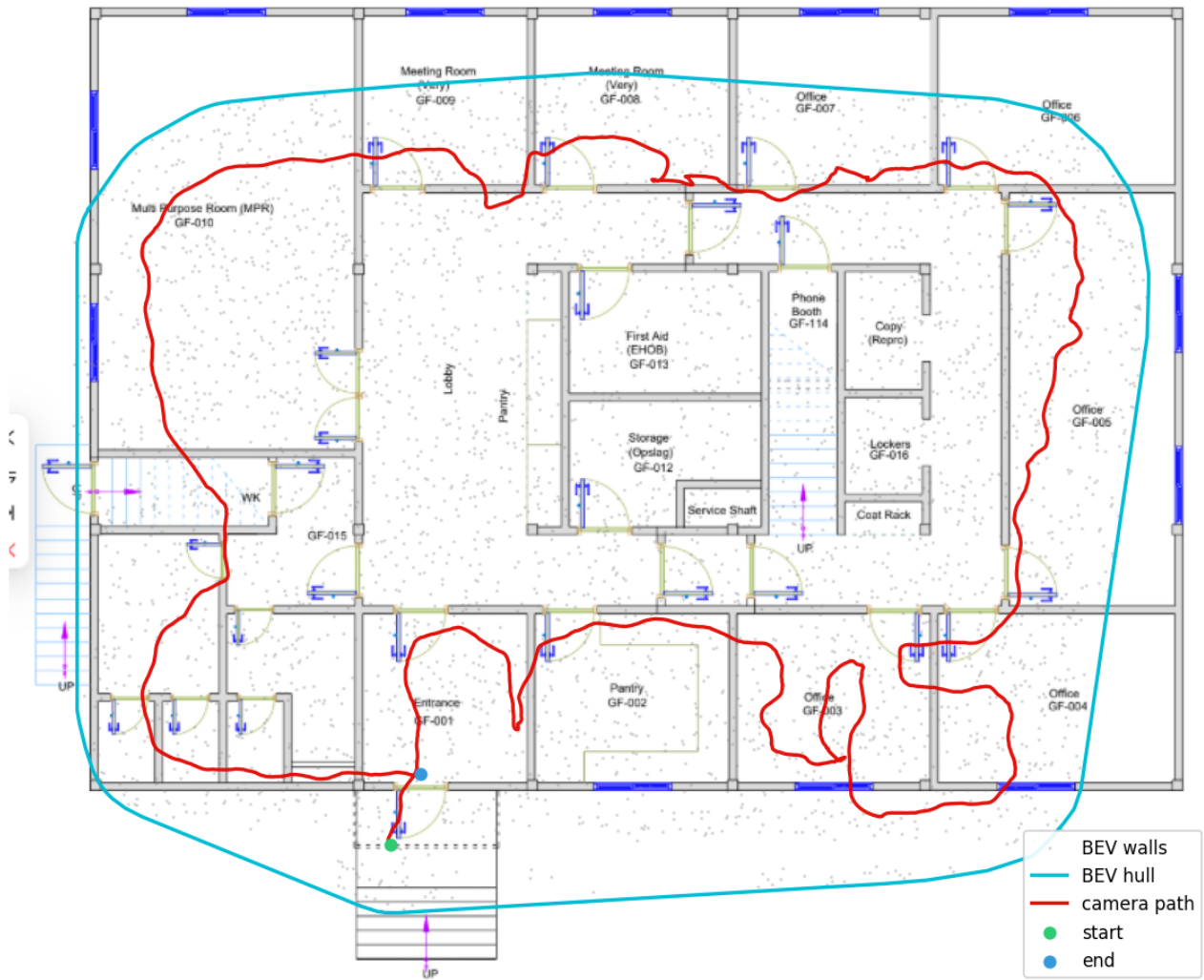
Metric	Hull Auto	3-Click Reference	Error
Scale	35.05 px/m	38.53 px/m	0.91x (9% low)
Rotation	-153.12°	-154.09°	0.97°
Translation	(244.4, 542.6)	(259.9, 519.1)	28.1 px drift
Free-space	0.907	0.932	-2.5%
Chamfer	0.263	—	—
Hull IoU	0.812	—	—

The rotation was surprisingly close ($< 1^\circ$ error), but the **scale was 9% too low** and **translation drifted 28 px**.

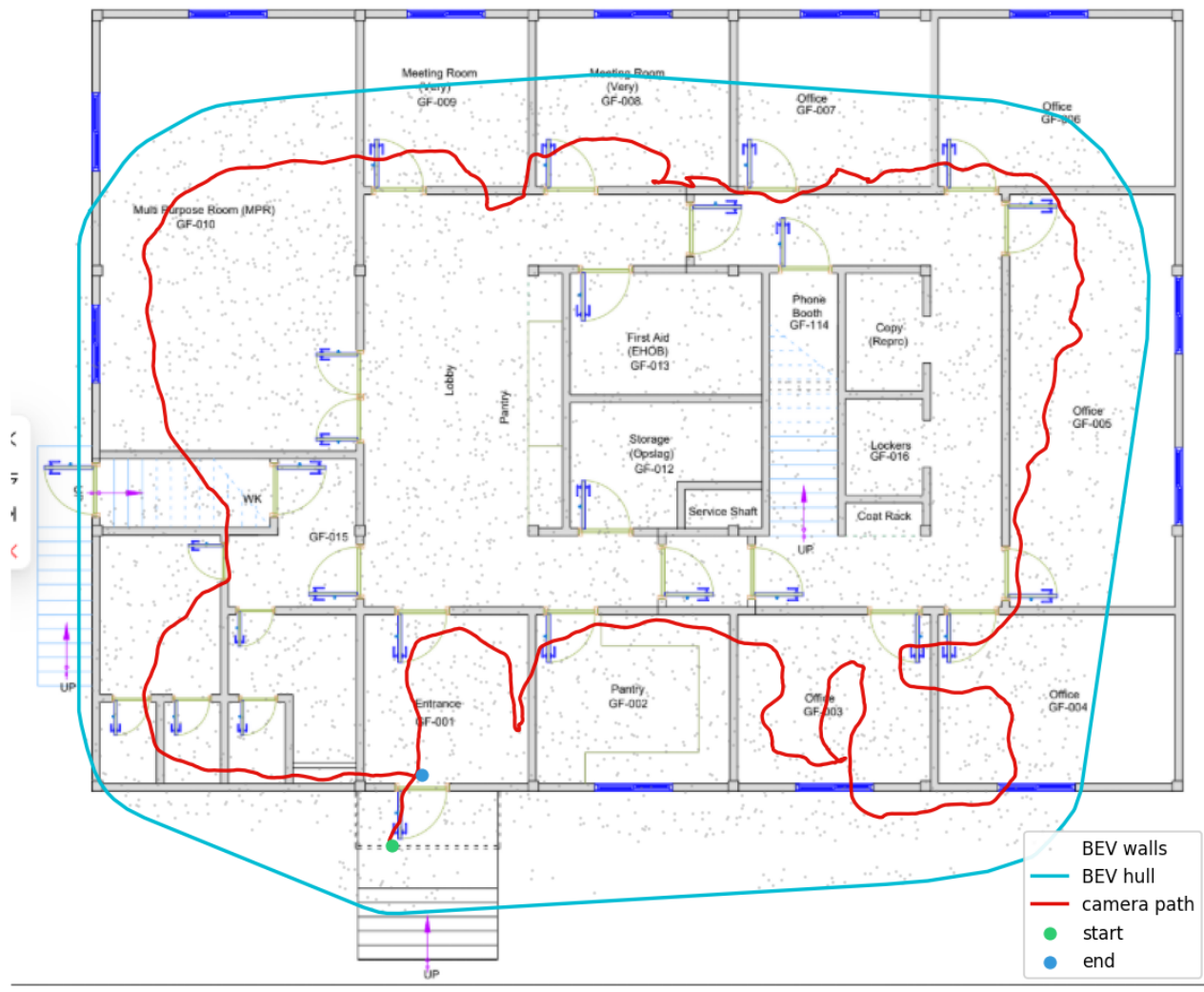




coarse hull score=0.566 dt=13.4px free=0.91



scan-match score=0.566 dt=13.4px free=0.91 iou=0.81



4. **Fundamental assumption violation:** Hull alignment assumes both hulls represent the same physical outline. This breaks on construction sites with incomplete walls and SLAM seeing beyond the building.

User feedback: *"The hull for my point cloud includes areas outside of the building... StellaVSLAM captures the exterior of the construction site as well, so I don't like this hull alignment approach."*

Lesson: Hull-based alignment requires clean, matching outer boundaries. On active construction sites, this assumption fails.

11. Phase 9: Start-Pinned Alignment

11.1 Motivation

After hull alignment failed, the user proposed: *"What if we mark our starting point on the floorplan ourselves?"*

Reduce the problem: fix translation with 1 click, optimise only scale and rotation geometrically.

11.2 Method

Script: `pipeline/src/07_start_fit_align.py`

1. **Anchor:** The first trajectory point is pinned to a user-specified floorplan coordinate (from the first pair in `correspondences.json`).
2. **Grid search (scale × rotation):**
 3. Scale: 20–60 px/m
 4. Rotation: 0°–360° in 1° steps
 5. For each (s, θ), construct Sim(2) with pinned translation
 6. Score = weighted sum of:
 - **Point chamfer:** mean distance of BEV wall points to nearest floorplan wall
 - **Line chamfer:** mean distance of sampled BEV line segment points to walls
 - **Free-space:** fraction of trajectory on walkable area
 - **Scale prior:** soft penalty pulling scale toward a reference value
7. **Local refinement:** Finer grid (0.2 px/m, 0.5°) around best coarse result.

11.3 Theoretical Justification

- 1 click fixes 2 DoF (translation), reducing search from 4D to 2D

- **Wall distance transform** provides smooth cost landscape for scale/rotation optimisation
- **Line segments** add rotational constraint (wall orientations must match)
- **Scale prior** prevents chamfer-driven collapse to small scales

11.4 Results

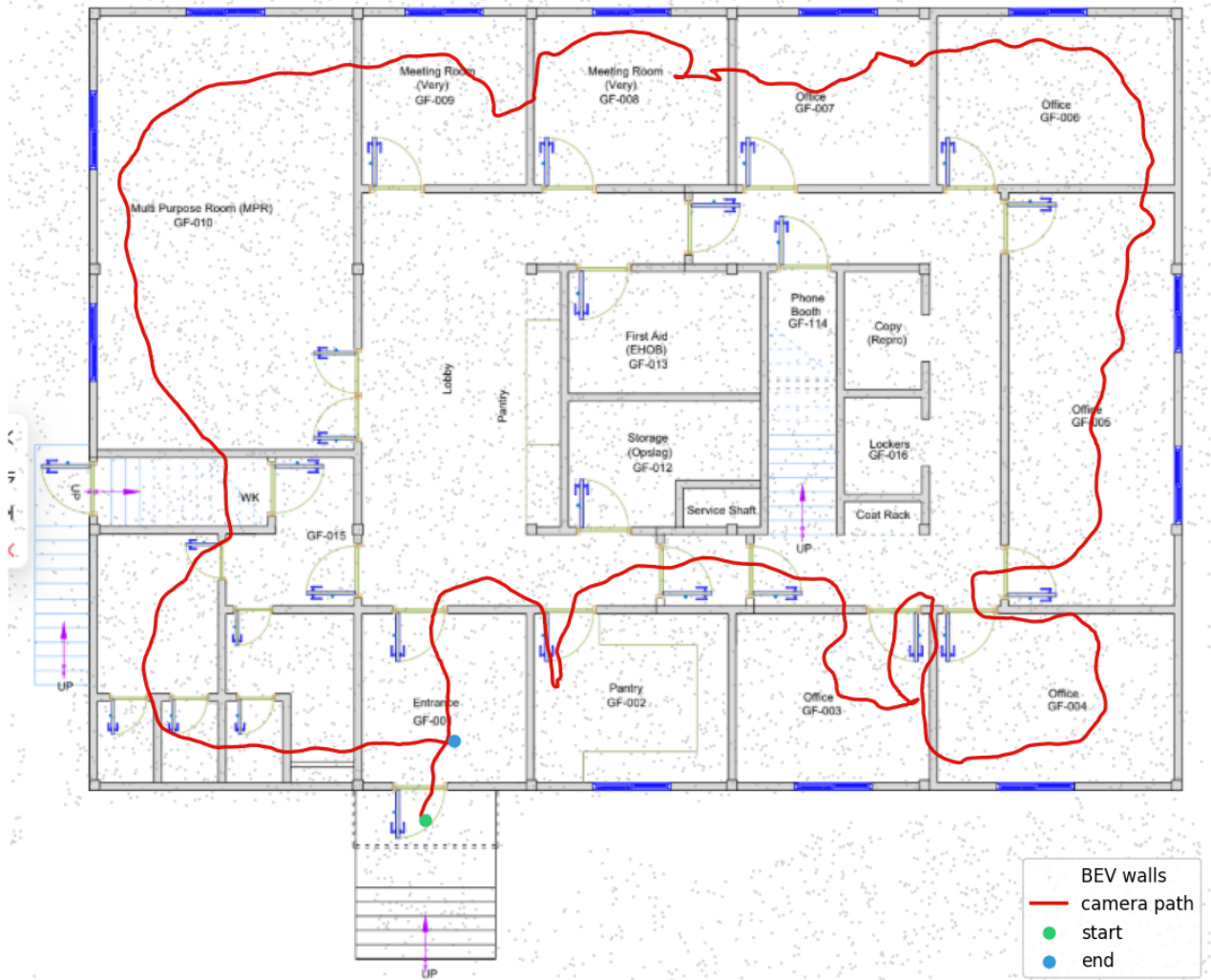
Output: `pipeline/outputs/run2-stella-start-fit/`

From `summary.json`, compared to 3-click reference:

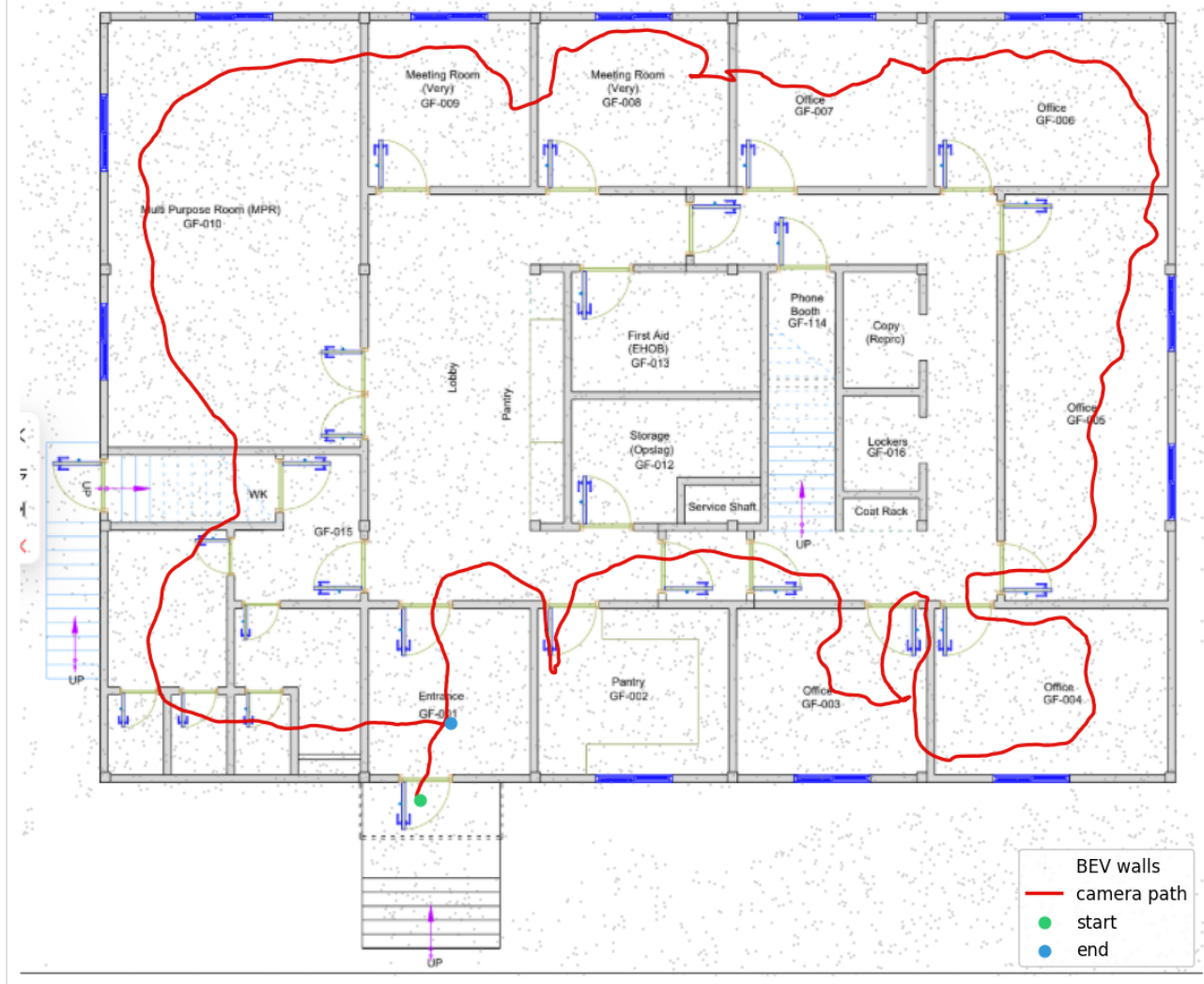
Metric	Start-Pinned	3-Click Reference	Error
Scale	38.50 px/m	38.53 px/m	< 0.1%
Rotation	-155.94°	-154.09°	1.85°
Translation	(266.4, 526.8)	(259.9, 519.1)	10.1 px
Free-space	0.932	0.932	identical
Point chamfer	0.106	—	—
Line chamfer	0.027	—	—

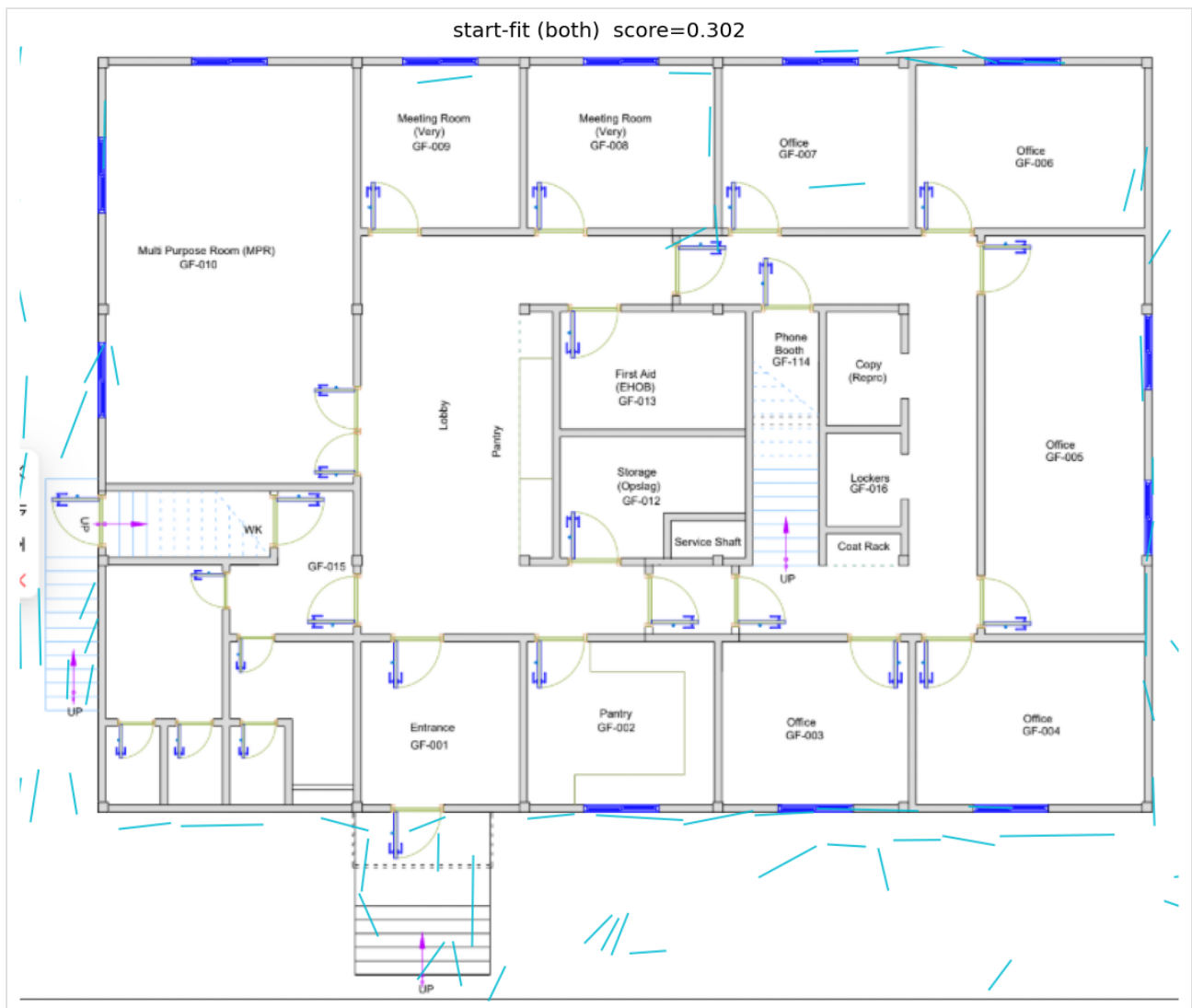
Scale is near-identical to reference (but uses a soft prior centred on the 3-click scale — acknowledged as a dependency). Rotation error is 1.85°. Translation error is 10 px (vs 28 px for hull approach). Free-space is identical.

Stella start_fit_both-align confidence=0.30



Stella click-align confidence=0.88





11.5 Current Limitations

1. **Scale prior dependency:** Uses the 3-click alignment's scale as prior — not truly 1-click independent. Needs an independent scale source (IMU step length, known wall dimension).
 2. **Rotation sensitivity:** 1° increments; for large buildings, even small angular error translates to meters of drift at trajectory ends.
 3. **Construction noise:** BEV wall points include debris and temporary structures not on floorplan.
 4. **This is "1-click start + geometry for rotation + 3-click scale as a cheat code"** — not yet a true 1-click solution.
-

12. Summary of All Approaches & Lessons Learned

Phase	Approach	Input	Scale Source	Map?	Result	Status
1	IMU PDR	IMU only	Step length (0.7m)	No	Drifty trajectory, no walls	Baseline
2	Classical VIO	Video + IMU	Optical flow calibration	Sparse occupancy	Noisy, neg free-space	Abandoned
3	MASt3R	Video	Metric model	No (camera poses only)	Good trajectory, no map	Moved on
4	StellaVSLAM	Video (equirect)	SLAM (up to scale)	Dense 3D → BEV	Ceiling noise dominates	Improved by mask
5	SAM3 Roof Mask	Video frames	—	—	Clean ceiling removal	Working
6	Stella + Mask + BEV	Video + mask	SLAM	Clean BEV	Clear wall outlines	Working
7	Manual Click-Align	3 user clicks	Umeyama from clicks	—	0.932 free-space, 9.3px RMSE	Working (manual)
8	Hull Auto-Align	None	Hull rectangle ratios	—	9% scale error, 28px drift	Abandoned
9	Start-Pinned	1 click + prior	Grid search + prior	—	< 0.1% scale, 1.85° rotation	In Progress

Key Lessons

1. **Start simple, escalate systematically.** Each phase was motivated by specific measured failures of the previous approach.
2. **IMU alone is insufficient** for indoor mapping. Heading drift and lack of visual features make PDR a trajectory-only tool.
3. **Classical VIO without optimisation accumulates fatal drift.** The absence of bundle adjustment, loop closure, or Kalman filtering means errors grow unboundedly.
4. **MASt3R gives good poses but no dense map.** For wall detection and BEV generation, you need a system that produces dense 3D points, not just camera positions.

5. **StellaVSLAM is excellent for 360-degree input** but produces noisy maps when ceilings are feature-rich. The ceiling masking step is essential.
 6. **SAM3 text-prompted segmentation works** for ceiling removal, but requires negative prompts, confidence tuning, and geometric sanity checks — especially on construction sites with missing walls exposing sky.
 7. **Hull-based alignment fails on incomplete structures.** It requires clean, matching outer boundaries — a condition that doesn't hold on active construction sites.
 8. **Reducing DoF is more robust than searching high-dimensional spaces.** Going from 4D (hull) to 2D (start-pinned) dramatically reduces ambiguity.
 9. **Scale is the hardest parameter to recover.** Rotation can be constrained by wall directions. Translation can be fixed by one click. Scale needs either a metric reference or multiple well-separated correspondences.
 10. **Distance transform chamfer scoring is effective** for local alignment but can produce degenerate solutions without regularisation (scale priors).
-

13. Appendix: Pipeline Architecture & Scripts

13.1 Original VIO Pipeline (src/)

Script	Purpose
<code>insv.py</code>	Parse Insta360 INSV binary container, extract IMU
<code>pdr.py</code>	Pedestrian dead reckoning from IMU (heading + steps)
<code>run_walk.py</code>	IMU extraction + PDR + floorplan overlay
<code>vio.py</code>	Classical VIO (optical flow + IMU heading)
<code>mast3r_odom.py</code>	MASt3R frame extraction + sparse global alignment
<code>mast3r_backend.py</code>	MASt3R backend wrapper
<code>backend.py</code>	Backend protocol + factory (<code>classical</code> / <code>mast3r</code>)
<code>align.py</code>	Sim(2) alignment via Umeyama + landmark correspondences
<code>floorplan.py</code>	Floorplan wall extraction + free-space scoring
<code>click_align.py</code>	Interactive click UI for landmark selection
<code>run_align.py</code>	Full pipeline: IMU → VIO/MASt3R → alignment → overlay
<code>optimize.py</code>	Wall-based trajectory refinement (post-alignment nudge)

13.2 StellaVSLAM Pipeline (pipeline/src/)

Script	Purpose
00_sam3_roof_mask.py	SAM3-based roof/ceiling mask generation
01_cubemap_crop.py	Equirectangular to cubemap face extraction
02_stella_slam.py	StellaVSLAM execution (Docker)
03_auto_align.py	Click-based Sim(2) alignment + BEV extraction
04_bev.py	BEV occupancy grid generation
05_walls.py	Wall line detection (Hough + morphology)
06_hull_scan_align.py	Hull-based auto alignment (abandoned)
07_start_fit_align.py	Start-pinned semi-auto alignment
run_pipeline.py	Full pipeline runner
smoke_test.py	End-to-end smoke test

13.3 Key Dependencies

- **SAM3**: Meta's Segment Anything 3 (Docker, PyTorch, sam3.pt)
- **StellaVSLAM**: Dense SLAM with equirectangular support (Docker, stella_vslam_dense)
- **MASt3R**: naver/MASt3R_ViTLarge_BaseDecoder_512_catmlpdpt_metric (PyTorch, CUDA)
- **OpenCV**: Image processing, optical flow, morphology, Hough lines, distance transform
- **NumPy/SciPy**: Numerical computation, signal processing
- **Matplotlib/PIL**: Visualisation and image I/O

13.4 Output Directory Structure

```
outputs/                                     # Original VIO/MASt3R outputs
├── VID_20260810_124225_00_064/              # Phase 1: IMU PDR
│   ├── imu_pdr.png
│   ├── trajectory_on_floorplan.png
│   ├── trajectory_vio.png
│   └── occupancy.png
├── run2-capture/                             # Phase 2: Classical VIO
│   ├── trajectory_aligned.png
│   ├── trajectory_vio.png
│   ├── occupancy.png
│   ├── walls.png
│   └── align_summary.json
└── run2-capture-mast3r/                     # Phase 3: MASt3R
```

```

├─ trajectory_aligned.png
├─ trajectory_vio.png
├─ mast3r_cams2world.npy
└─ vio_meta.json

pipeline/outputs/                                # StellaVSLAM pipeline outputs
├─ run2-stella/                                  # Phase 4: Stella (no mask)
│   ├─ bev.png, walls.png
│   └─ roof_mask.png
├─ run2-stella-roofmask/                         # Phases 5-7: Stella + mask + alignment
│   ├─ bev.png, walls.png
│   ├─ roof_mask.png, roof_mask_overlay.jpg
│   ├─ sim2_transform.json
│   ├─ correspondences.json
│   └─ auto-align/                               # Phase 8: Hull alignment
│       ├─ plan_hull.png, bev_hull.png
│       ├─ overlay_coarse.png, overlay_fine.png
│       └─ trajectory_aligned.png
└─ run2-stella-start-fit/                       # Phase 9: Start-pinned
    ├─ trajectory_aligned.png
    ├─ overlay_click_reference.png
    └─ lines_overlay.png

```